

Chapitre 2 : Arbres de Décision (CART)

Apprentissage Supervisé

Dr Seydou Nourou SYLLA

Université Alioune DIOP de Bambey
Master SI/SR

2 mars 2026

Plan du chapitre

- 1 Intuition et structure
- 2 Critères de coupure : Entropie et Gini
- 3 Algorithme CART from scratch
- 4 Élagage et régularisation
- 5 Interprétabilité et importance des features

L'intuition derrière les arbres de décision

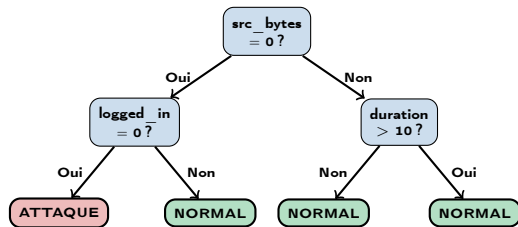
Analogie : Un arbre de décision mime le raisonnement humain par **questions successives**.

Pour l'exemple d'application 1 :

- ❶ $\text{src_bytes} = 0 ?$
 - Oui $\rightarrow \text{logged_in} = 0 ?$
 - Non $\rightarrow \text{duration} > 10 ?$
- ❷ ... jusqu'à une décision finale

Avantages de la représentation en arbre :

- **Interprétable** par un expert sécurité
- Capture les **non-linéarités**
- Gère les variables **hétérogènes**



Mesurer l'impureté d'un noeud

Objectif : Trouver la meilleure question (feature + seuil) pour diviser les données.

On mesure l'**impureté** d'un nœud contenant n exemples avec n_c exemples de classe c :

$$p_c = \frac{n_c}{n}$$

Entropie de Shannon :

$$H = - \sum_{c=1}^K p_c \log_2(p_c)$$

- $H = 0$: nœud pur (une seule classe)
- $H = 1$: nœud 50/50 (binaire)
- Utilisée dans ID3, C4.5

Indice de Gini :

$$G = 1 - \sum_{c=1}^K p_c^2 = \sum_{c \neq c'} p_c p_{c'}$$

- $G = 0$: nœud pur
- $G = 0.5$: nœud 50/50 (binaire)
- Utilisé dans **CART** (sklearn)

Calcul du gain d'information

Meilleure coupure = celle qui **réduit le plus l'impureté** après la division.

Soit une coupure sur la feature j au seuil t , produisant sous-ensembles $\mathcal{L}$ (gauche) et $\mathcal{R}$ (droite) :

$$\text{Gain}(j, t) = I(\text{parent}) - \frac{|\mathcal{L}|}{n} I(\mathcal{L}) - \frac{|\mathcal{R}|}{n} I(\mathcal{R})$$

où $I(\cdot)$ est l'entropie ou l'indice de Gini.

Exemple d'application 1 (src_bytes = 0 ?) :

Nœud	Normal	Attack	Gini
Parent (100 ex.)	50	50	$1 - (0.5^2 + 0.5^2) = 0.50$
Gauche (src=0, 60 ex.)	5	55	$1 - (0.083^2 + 0.917^2) = 0.152$
Droite (src>0, 40 ex.)	45	5	$1 - (0.875^2 + 0.125^2) = 0.219$
Gain	$0.50 - \frac{60}{100} \times 0.152 - \frac{40}{100} \times 0.219 = 0.297$		

Algorithme CART – Pseudocode récursif

Algorithme 1 : CART – Construction récursive

Input : Dataset $\mathcal{D}$, profondeur max d_{max} , taille min feuille n_{min}

Output : Nœud racine de l'arbre

```
if  $|\mathcal{D}| < n_{min}$  OU profondeur  $\geq d_{max}$  OU  $\mathcal{D}$  est pur then
    return feuille avec classe majoritaire de  $\mathcal{D}$ ;
else
    foreach feature  $j$  et seuil  $t$  do
        Calculer  $\text{Gain}(j, t)$  ;
    end
     $(j^*, t^*) \leftarrow \arg \max_{j, t} \text{Gain}(j, t)$ ;
     $\mathcal{L} \leftarrow \{x \in \mathcal{D} : x_j \leq t^*\}$ ;
     $\mathcal{R} \leftarrow \{x \in \mathcal{D} : x_j > t^*\}$ ;
    return nœud( $j^*, t^*, \text{CART}(\mathcal{L}), \text{CART}(\mathcal{R})$ );
end
```

Overfitting et élagage (pruning)

Problème : Un arbre non contraint mémorise le train.

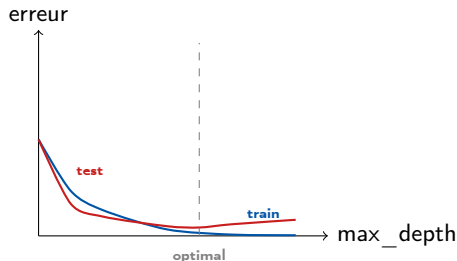
Hyperparamètres de régularisation :

- `max_depth` : profondeur maximale
- `min_samples_split` : taille minimale pour couper
- `min_samples_leaf` : taille minimale d'une feuille
- `max_features` : nb features testées par nœud
- `ccp_alpha` : élagage post-construction (cost-complexity)

Élagage cost-complexity (CART) :

$$R_{\alpha}(T) = R(T) + \alpha \cdot |T|$$

où $|T|$ est le nombre de feuilles et α pénalise la complexité.



Idée : Une feature est importante si elle génère beaucoup de gain d'information.

$$\text{Importance}(j) = \sum_{\text{nœuds } t \text{ où } j \text{ est utilisé}} \frac{n_t}{n} \cdot \text{Gain}(t)$$

Avantages de l'interprétabilité (ex. application 1)

L'arbre de décision révèle la logique métier :

- $\text{src_bytes} = 0 \wedge \text{logged_in} = 0 \Rightarrow$ **ATTAQUE**
 - Cohérent avec les scans de ports
- $\text{dst_bytes} > 50000 \wedge \text{logged_in} = 1 \Rightarrow$ **NORMAL**
 - Sessions de téléchargement légitimes

Cas d'usage professionnel :

- Audit de sécurité : expliquer les décisions
- Validation par les experts
- Extraction de règles de firewall

Avantage clé

L'arbre de décision est le seul modèle du cours qui s'explique **nativement** en langage naturel.

Rappel

KNN est une boîte noire : on ne comprend pas *pourquoi* il classe ainsi, juste *qu'il y a des voisins similaires*.

À retenir

- CART divise récursivement selon le meilleur **gain d'impureté** (Gini/Entropie)
- Aucune normalisation requise, gère les features hétérogènes
- Très interprétable mais sujet à l'overfitting sans régularisation
- Performances excellentes sur l'exemple d'application 1 ($> 99.6\%$) avec bonne profondeur

Hyperparamètre	Valeur recommandée (ex. app. 1)
max_depth	10 (validation croisée)
min_samples_split	5–20
criterion	gini (légèrement plus rapide)